

Design Document: Host Inventory V2 API

Service	Host Inventory
Jira/Ticket	RHINENG-13956
Author	Muhammad Arif
Status	Draft
Proposed date	Jun 9, 2026
Accepted date	

1. Executive Summary

The Host Inventory (HBI) REST API currently serves all endpoints under `/api/inventory/v1`. Three feature domains cannot be added to V1 without breaking existing consumers: host application data enrichment (host_apps), workspaces (evolution of groups), and saved host-view configurations. This document proposes moving the entire API surface to `/api/inventory/v2` with a dedicated OpenAPI spec (`api_v2.spec.yaml`) and isolated controller layer (`api/v2/`), while V1 continues to run unchanged alongside it.

2. Problem Statement

The current single-version architecture creates several constraints:

- **No place for host_apps data:** Six host-returning endpoints cannot include application data (advisor, vulnerability, patch, compliance, remediations, malware) without changing the V1 response contract that downstream consumers depend on.
- **Groups schema is frozen:** The V1 `GroupOut` schema has no `parent_id`, `description`, or `type` fields, and the timestamp is named `updated`. Adding fields or renaming would break V1 consumers.
- **Host-views stuck in beta:** Seven endpoints under `/beta/` (one implemented, six stubs) have no permanent versioned home.
- **Single spec coupling:** All operations share one 100KB OpenAPI spec. Changes to support new response shapes ripple across the entire spec, increasing review burden and regression risk.

3. Proposed Architecture: Isolated V2 Controller Layer

All endpoints move from V1 to V2. V2 runs as a separate Connexion API registration with its own OpenAPI spec and controller modules. V1 remains untouched and runs alongside V2.

3.1 Routing

```
V1: /api/inventory/v1/* → swagger/api.spec.yaml → api/*.py
(unchanged)
V2: /api/inventory/v2/* → swagger/api_v2.spec.yaml → api/v2/*.py
(new)
```

V2 registers via `RestyResolver("api.v2")` in `app/__init__.py`, resolving operations to `api/v2/*.py`.

3.2 Shared Infrastructure

V2 controllers call the same business logic in `lib/` and use the same SQLAlchemy models in `app/models/`. No database schema changes are required for the routing layer.

3.3 Changes by Category

Category	Count	V2 Change
Host_apps enrichment	6	Host responses add a <code>host_apps</code> object with data from six application tables
Workspaces	5	Groups → Workspaces: add <code>parent_id</code> , <code>description</code> , <code>type</code> ; <code>rename</code> updated → <code>modified</code>
Host-views	7	<code>GET /beta/hosts-view</code> absorbed into <code>GET /v2/hosts</code> ; <code>/beta/views/*</code> moves to <code>/v2/host-views/*</code>
Unchanged	23	Identical request/response to V1 (deletes, tags, staleness, <code>system_profile</code> , facts, resource-types)

3.4 Host_apps Enrichment

V2 host responses are a superset of V1. The existing `HostAppData*` models (`app/models/host_app_data.py`) store per-host data from six applications across six tables:

`hosts_app_data_advisor`, `hosts_app_data_vulnerability`, `hosts_app_data_patch`, `hosts_app_data_remediations`, `hosts_app_data_compliance`, `hosts_app_data_malware`. V2 host responses include a `host_apps` object containing serialized data from these tables.

The join logic already exists in `api/host_views.py:_fetch_app_data_for_hosts()`. V2 reuses this function in all six host-returning endpoints.

Affected operations:

Method	V2 Path
GET	<code>/hosts</code>
GET	<code>/hosts/{host_id_list}</code>
POST	<code>/hosts/checkin</code>
PATCH	<code>/hosts/{host_id_list}</code>
GET	<code>/hosts/{host_id_list}/system_profile</code>
GET	<code>/workspaces/{workspace_id}/hosts</code>

3.5 Workspaces

V2 renames “groups” to “workspaces” at the API level. The underlying `Group` model (`app/models/group.py`) gains three columns via Alembic migration:

Column	Type	Notes
<code>parent_id</code>	UUID, nullable	Self-referential FK to <code>hbi.groups.id</code>
<code>description</code>	Text, nullable	Free-text description
<code>type</code>	String, nullable	Workspace classification

The response field `updated` is renamed to `modified` in V2 serialization. The DB column is already named `modified_on`, so this is a serialization change only.

Affected operations:

Method	V1 Path	V2 Path
GET	<code>/groups</code>	<code>/workspaces</code>
POST	<code>/groups</code>	<code>/workspaces</code>
PATCH	<code>/groups/{group_id}</code>	<code>/workspaces/{workspace_id}</code>
GET	<code>/groups/{group_id_list}</code>	<code>/workspaces/{workspace_id_list}</code>
POST	<code>/groups/{group_id}/hosts</code>	<code>/workspaces/{workspace_id}/hosts</code>

3.6 Host-Views

Two changes happen here:

1. **GET /beta/hosts-view is absorbed into GET /v2/hosts**. Since all V2 host endpoints return `host_apps` data, the dedicated `hosts-view` endpoint is redundant. The sparse field selection (`?fields[host_apps]=advisor,patch`) from `api/host_views.py` moves into the standard host query.
2. **/beta/views/* moves to /v2/host-views/***. The `InventoryView` model and migration already exist (PR #4128). The six 501 stubs in `api/views.py` are implemented as full controllers.

Method	V1 Path	V2 Path	Current Status
GET	/beta/views	/host-views	501 stub
POST	/beta/views	/host-views	501 stub
GET	/beta/views/{view_id}	/host-views/{view_id}	501 stub
PUT	/beta/views/{view_id}	/host-views/{view_id}	501 stub
DELETE	/beta/views/{view_id}	/host-views/{view_id}	501 stub
POST	/beta/views/{view_id}/clone	/host-views/{view_id}/clone	501 stub

4. Implementation Plan

Phase 1: V2 Routing Skeleton

Stand up the V2 API so all endpoints are reachable at `/api/inventory/v2/`.

- **OpenAPI Spec:** Create `swagger/api_v2.spec.yaml` derived from `api.spec.yaml`. Update all `operationId` values to `api.v2.*` namespace. Remove `/beta/` paths. Add `/host-views/*` paths. Rename `/groups` paths to `/workspaces`.
- **Connexion Registration:** Add V2 API registration in `app/__init__.py` with `RestyResolver("api.v2")` and `_build_v2_api_path()` returning `/api/inventory/v2`.
- **Config:** Add `v2_api_url_path_prefix` to `app/config.py`, append to `api_urls`.
- **Controller Package:** Create `api/v2/` with modules mirroring V1. For the 23 unchanged endpoints, V2 controllers delegate directly to existing `lib/` functions.

Phase 2: Host_apps & Workspaces

- **Host_apps Serialization:** Create V2 host serialization that calls `_fetch_app_data_for_hosts()` and merges the `host_apps` object into host responses. Support sparse field selection via `?fields[host_apps]=advisor,patch.`
- **Workspaces Migration:** Alembic migration adding `parent_id`, `description`, and `type` to `hbi.groups`. All nullable, no table rewrite.
- **Workspaces Serialization:** V2 workspace serializer maps `modified_on` → `modified` and includes the three new fields.
- **Workspaces Controllers:** `api/v2/group.py` handles `/workspaces` paths using the `Group` model with the new serializer.

Phase 3: Host-Views Implementation

- **Repository Layer:** Create `lib/view_repository.py` with CRUD operations for `InventoryView`, including visibility rules (system views readable by all, org views by org, private views by creator).
- **Controllers:** Implement `api/v2/views.py` (list, create, get, update, delete, clone).
- **RBAC:** Register `inventory-views` resource type in `app/auth/rbac.py`.

5. Performance and Resource Impact

Metric	Impact	Details
Latency (unchanged endpoints)	None	V2 delegates to same <code>lib/</code> functions as V1
Latency (host_apps)	Moderate	LEFT JOINS to 6 <code>hosts_app_data_*</code> tables; mitigated by sparse field selection and existing indexed FKs
Latency (workspaces)	Minimal	Three additional columns serialized; no new joins
Storage	Minimal	Three new nullable columns on <code>hbi.groups</code>

6. Risk Assessment and Mitigation

Risk	Severity	Mitigation
V1 regression	High	V1 code and spec are not modified. V2 is a separate Connexion registration with its own controllers.
Host_apps query performance	Medium	Sparse field selection (<code>?fields[host_apps]=advisor</code>)

Risk	Severity	Mitigation
		limits JOINS to requested app tables only.
Workspaces migration	Low	<code>hbi.groups</code> is not partitioned. Adding nullable columns is a metadata-only operation.
View ownership bugs	Medium	Integration tests for each visibility level: system views, org-wide views, private views.
Spec drift	Low	V2 spec is standalone. Drift from V1 is intentional — V2 evolves independently.